

to the most effective ones.

Model selection

```
from sklearn.ensemble import
RandomForestClassifier
from sklearn.svm import SVC

def select_model(model_type):
    if model_type == 'RandomForest':
        return RandomForestClassifier()
    elif model_type == 'SVM':
        return SVC()
```

Hyperparameter tuning

```
def tune_hyperparameters(model,
hyperparameters):
    model.set_params(**hyperparameters)
    return model
```

Fitness function

```
from sklearn.metrics import
accuracy_score

def fitness_function(model, X_train,
y_train, X_test, y_test):
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    return accuracy_score(y_test,
predictions)
```

Genetic algorithms can automate the selection and tuning process by exploring the model and hyperparameter space efficiently. The GA will evaluate the performance of each candidate solution (combination of model and hyperparameters) using a fitness function—in this case, the model's accuracy score.

The end-to-end pipeline

The ultimate goal is to bring all these individual pieces into a coherent

whole—an end-to-end pipeline that takes raw data and outputs an optimised machine learning model. In this pipeline, genetic algorithms play a pivotal role in automating multiple steps, from data preparation to model tuning.

End-to-end pipeline

```
from sklearn.model_selection import
train_test_split
```

```
def end_to_end_pipeline(raw_data,
target, model_type='RandomForest'):
```

Step 1: Data preparation

```
    clean_data = handle_missing_
values(raw_data)
    normalized_data = normalise(clean_
data)
```

Step 2: Feature selection

```
    X_train, X_test, y_train, y_test
= train_test_split(normalized_data,
target, test_size=0.2)
    selected_features = [i for i in
range(len(X_train[0]))] # Placeholder,
would be determined by GA
```

Step 3: Model selection and tuning

```
    model = select_model(model_type)
    hyperparameters = {} # Placeholder,
would be determined by GA
    tuned_model = tune_
hyperparameters(model, hyperparameters)
```

Step 4: Evaluate fitness

```
    fitness = fitness_function(tuned_model,
X_train[:, selected_features], y_train,
X_test[:, selected_features], y_test)
```

```
    return fitness
```

Example usage

```
raw_data = np.random.rand(100, 10)
# 100 samples, 10 features
target = np.random.randint(0, 2, 100)
# Binary target variable

fitness = end_to_end_pipeline(raw_data,
target)
```

This is a simplified example, but it gives you a blueprint for constructing an end-to-end pipeline that employs genetic algorithms at every key stage. This ensures that you're extracting the most value from your data at each step of the machine learning process.

From automating the tedious process of data preparation to fine-tuning machine learning models, genetic algorithms provide an efficient, automated approach to optimise the entire data pipeline. By leveraging these algorithms, we're not just simplifying the model development process but also ensuring that the highest quality insights are gleaned from our data. Whether dealing with large-scale data sets, multi-dimensional features, or diverse machine learning models, genetic algorithms equip you with the versatility to handle a broad array of data challenges. As a result, they become an indispensable asset in any data scientist's toolkit for crafting robust and effective solutions. **EFY**

This article was first published in December 2023 issue of Open Source For You magazine.

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU